

DOSSIER ÉCRIT DE CERTIFICATION

Bloc 1 — Collecter, transformer et sécuriser des données

Titre visé : Ingénieur en science des données spécialisé en infrastructure data ou en apprentissage automatique — RNCP 39586 (niveau 7)

Projet : Solution de gouvernance et d'archivage intelligent des données d'un ERP PeopleSoft (Oracle EP92U038)

Épreuve écrite individuelle — dossier de 20 pages maximum, hors annexes et page de garde
[Lien Github](#)

Candidat : HAOUARI Salem

Mastère 2 — filière IA & Data · YNOV

Date : 14/07/2026

***Note de lecture pour le jury.** Chaque section correspond à une compétence du bloc (C1.1.1 → C1.4.2). Elle présente le **livrable attendu** puis se termine par un encadré « **Critères couverts** » reprenant les exigences de la grille. Les références de code renvoient au dépôt Git du projet.*

Sommaire

1. Contexte, problématique et périmètre.....	2
2. C1.1 — Collecter les données	3
• C1.1.1 Stratégie de collecte.....	3
• C1.1.2 Exemple de collecte et techniques employées	4
• C1.1.3 Automatisation de la collecte	6
3. C1.2 — Stocker les données	7
• C1.2.1 Stratégie de stockage et modèle de données.....	7
• C1.2.2 Construction de la base de données.....	9
4. C1.3 — Transformer les données	11
• C1.3.1 Sélection des technologies de traitement	11
• C1.3.2 Transformation des données	11
• C1.3.3 Processus ETL et orchestration	12
5. C1.4 — Sécuriser les données	13
• C1.4.1 Politique de sécurité des données	13
• C1.4.2 Architecture de sécurité.....	15
6. Conclusion	17

1. Contexte, problématique et périmètre

1.1 Problématique

L'entreprise exploite un ERP **Oracle PeopleSoft (instance EP92U038)** dont la base a fortement grossi au fil des années. Le stockage de données historiques inactives sur du stockage « chaud » coûte cher, ralentit les sauvegardes et complexifie la conformité (RGPD, durées légales de rétention). L'entreprise a besoin d'un outil capable de **cartographier son patrimoine de données, d'en identifier le domaine métier, et de recommander une stratégie d'archivage par table**, sans jamais mettre en péril l'intégrité du SI ni la conformité légale.

Le premier maillon de cette chaîne — et l'objet de ce bloc — est de **collecter de façon fiable et reproductible les métadonnées** de l'ERP, de les **stocker dans un entrepôt structuré**, de les **transformer en un modèle exploitable** pour l'analyse et le Machine Learning, et de **sécuriser** l'ensemble de la chaîne.

1.2 Sources de données

Source	Nature	Volume constaté	Rôle
Oracle PeopleSoft EP92U038	Base relationnelle (dictionnaire de données applicatif + tables physiques)	87 627 records logiques · 79 633 tables physiques (schéma SYSADM) · 107 206 champs · PeopleTools 8.58	Source unique : structure et volumétrie du patrimoine

1.3 Cible

Un entrepôt **PostgreSQL** (`pfe\_data\_ia`) organisé en **architecture médaille** (couches `raw` → `processed` → `serving`). Au terme d'un cycle complet (run 6), le catalogue consolidé réunit **≈ 167 260 actifs** de données (records logiques PeopleSoft et tables physiques Oracle).

1.4 Contraintes et points de vigilance

- **Réseau** : la source Oracle (`192.168.11.111`, IP privée) n'est accessible que par **VPN** — sessions longues sujettes aux coupures.
- **Confidentialité** : les métadonnées peuvent contenir des données personnelles (emails, identités d'opérateurs dans les logs) → enjeu RGPD.
- **Intégrité** : aucune écriture ne doit être faite sur la source (accès **lecture seule**).
- **Reproductibilité** : la collecte doit être **rejouable à l'identique** et **traçable** (audit, débogage).

2. C1.1 — Collecter les données

C1.1.1 — Stratégie de collecte de données

Livrable : une stratégie de collecte.

Objectifs de la collecte

Constituer un **catalogue exhaustif et fiable des métadonnées** du SI PeopleSoft permettant, en aval, de : (1) cartographier le patrimoine par domaine métier, (2) mesurer la volumétrie et le coût de stockage, (3) alimenter un modèle de classification ML, (4) produire des recommandations d'archivage.

Données utiles et nécessaires

On ne collecte **pas les données métier elles-mêmes** (contenu des tables) mais leurs **métadonnées** :

- **Structure logique** : records PeopleSoft (`PSRECDEFN`), champs (`PSDBFIELD`), associations record-champ (`PSRECFLDDB`).
- **Structure physique** : tables Oracle (`ALL\_TABLES`), colonnes (`ALL\_TAB\_COLUMNS`), index (`ALL\_INDEXES`), volumétrie (`num\_rows`, `blocks`, `avg\_row\_len`), dernière modification (`ALL\_TAB\_MODIFICATIONS`).
- **Signaux d'usage** : logs d'accès/segments (activité récente d'une table → candidat à l'archivage).

Sources de données

Décrite en §1.2 : Oracle EP92U038 (dictionnaire de données PeopleSoft).

Moyens envisagés

- **Connecteur dédié** en Python (``oracledb`` en mode `*thin*`) — voir C1.1.2.
- **Extraction incrémentale par préfixe** pour les gros volumes (tolérance aux coupures VPN, reprise sur incident).
- **Orchestration Prefect** pour l'automatisation et la traçabilité (voir C1.1.3).
- **Idempotence par ``run_id``** : chaque exécution est un instantané versionné et rejouable.

***Critères couverts (C1.1.1).** La stratégie identifie ✓ les **objectifs** de la collecte, ✓ les **données utiles et nécessaires** (métadonnées structurales + usage + enrichissement), ✓ la **source** (Oracle EP92U038), ✓ les **moyens** (connecteur Python, orchestration Prefect, extraction incrémentale idempotente).*

C1.1.2 — Exemple de collecte de données

Livrable : un exemple de collecte + démonstration de la qualité (exhaustivité, exactitude).

Techniques de collecte employées

Technique	Mise en œuvre dans le projet	Module
Requêtes SQL	Extraction du dictionnaire Oracle via des SELECT sur <code>`PSRECDEFN`</code> , <code>`ALL_TABLES`</code> , <code>`ALL_TAB_COLUMNS`</code> , <code>`ALL_INDEXES`</code> ...	<code>`src/extract/oracle/*`</code>
Accès base de données via driver natif	Lecture programmatique via le pilote <code>`oracledb`</code> (mode <code>*thin*</code> , protocole Oracle Net) — API native du SGBD, avec batching et résilience VPN	<code>`src/connectors/oracle_client.py`</code>
API externe	Appel de l' API publique Azure Retail Prices (sans clé) → tarifs réels du stockage Hot / Cool / Archive, convertis en <code>`\$/Go/an`</code> et injectés dans <code>`serving.dim_cost_params`</code>	<code>`src/extract/external/fetch_cloud_storage_prices.py`</code>
Web crawling / scraping	Extraction du tarif d'un fournisseur concurrent depuis sa page web publique (Backblaze B2), après vérification du <code>`robots.txt`</code> → benchmark de marché	<code>`src/extract/web/scrape_storage_benchmark.py`</code>

Pourquoi une API externe et du scraping dans un projet d'archivage ?

Le moteur de recommandation repose sur un **calcul de ROI** : archiver n'a de sens que si l'écart de coût entre stockage chaud et archive est réel. Or ces coûts étaient initialement des **constantes codées en dur** — une faiblesse identifiée dans notre propre évaluation de maturité. Les deux techniques y répondent :

- **API officielle (Azure)** — tarifs de référence faisant autorité : `Hot LRS` **0,2120 \$/Go/an**, `Archive LRS` **0,0216 \$/Go/an**, soit un **écart ×9,8** (gain de **0,19 \$/Go/an** par Go archivé).
- **Scraping (Backblaze B2)** — recoupement concurrentiel : **0,12 \$/Go/an**, ce qui établit une **fourchette de marché de 0,0216 à 0,12 \$/Go/an** pour l'archivage, au lieu d'un prix unique non sourcé.

Les tarifs de `roi\_score` proviennent donc désormais de **sources externes réelles et rejouables**, et non plus d'hypothèses.

Détail technique — SSL en environnement d'entreprise.** Le proxy de l'entreprise inspecte le trafic TLS, ce qui fait échouer la vérification de certificat standard. Plutôt que de désactiver la vérification (`verify=False`, faute de sécurité inacceptable dans un projet qui traite de sécurité), les modules utilisent `truststore` : la vérification SSL s'appuie sur le **magasin de certificats du système d'exploitation**, où la CA du proxy est déjà installée. **La vérification SSL reste donc active.

***Limites assumées du scraping.** L'extraction dépend de la structure HTML de la page cible : si le fournisseur modifie sa mise en page, le module **échoue explicitement** (jamais silencieusement) et lève une erreur explicite. Le scraping est donc cantonné à un rôle de **recoupement** — jamais de source unique de vérité. La collecte est en outre **responsable** : `robots.txt` vérifié avant toute requête, une seule requête par exécution, User-Agent explicite, et **aucune donnée personnelle** collectée.*

Exemple concret : extraction du catalogue des records logiques

Le module `src/extract/oracle/extract\_record\_catalog.py` exécute une requête paramétrée qui **joint la structure logique (PeopleSoft) à la structure physique (Oracle)** — jointure indispensable car PeopleSoft ne garantit aucune cohérence automatique entre les deux :

```
SELECT r.recname, r.recdescr, r.rectype,
       CASE WHEN TRIM(r.sqltablename) IS NOT NULL AND TRIM(r.sqltablename) <> ''
            THEN TRIM(r.sqltablename) ELSE 'PS_' || r.recname END AS physical_table_name,
       t.owner AS oracle_owner, t.table_name AS oracle_table_name,
       CASE WHEN t.table_name IS NOT NULL THEN 1 ELSE 0 END AS table_exists_flag
FROM SYSADM.PSRECDEFN r
LEFT JOIN ALL_TABLES t
      ON t.owner = :owner AND t.table_name = /* nom physique reconstruit */
WHERE r.recname <> 'PSDUMMY'
ORDER BY r.recname
```

Le connecteur (`src/connectors/oracle\_client.py`) matérialise chaque ligne en dictionnaire Python, lit les LOB tant que la connexion est ouverte, et applique un **batching** (`arraysize = prefetchrows = 2000`) pour limiter les allers-retours réseau.

Démonstration de la qualité de la collecte

Exhaustivité — les comptages de contrôle exécutés sur la source confirment la couverture complète du périmètre :

Contrôle	Valeur relevée
Records logiques collectés (hors `PSDUMMY`)	87 627
Tables physiques (schéma SYSADM)	79 633
Champs (`PSDBFIELD`)	107 206

Exactitude — trois mécanismes garantissent la fidélité des données collectées :

1. **Empreinte SHA-256** calculée sur chaque lot extrait (détection de toute altération entre extraction et chargement).
2. **`table\_exists\_flag`** : réconciliation logique↔physique (une définition de record sans table physique réelle est marquée, pas silencieusement perdue).
3. **Typage et normalisation** : noms de colonnes en minuscules, valeurs nulles explicitées (`NULLIF`, `TRIM`), LOB lus intégralement — évite les troncatures.

***Critères couverts (C1.1.2).** Un exemple de collecte est présenté (catalogue des records). Les quatre techniques exigées par la grille sont mises en œuvre : ✓ **requêtes SQL** (dictionnaire Oracle), ✓ **API externes** (Azure Retail Prices → `dim_cost_params`), ✓ **web scraping** (page tarifaire Backblaze B2), ✓ **web crawling responsable** (robots.txt vérifié avant requête) — auxquelles s'ajoute l'accès par driver natif (oracledb). La qualité est démontrée sur les deux axes exigés : ✓ **exhaustivité** (comptages de couverture, écart nul) et ✓ **exactitude** (hash SHA-256, réconciliation logique↔physique, normalisation).*

C1.1.3 — Automatisation de la collecte

Livrable : une méthode d'automatisation de la collecte.

L'automatisation repose sur l'orchestrateur **Prefect**. Chaque étape de collecte est une **task** ; les tâches sont assemblées en **flows** rejouables.

Composants de l'automatisation

- **Workflows (flows Prefect)** — ``src/prefect/flows/`` :
- ``flow_oracle_raw`` — collecte des 5 catalogues Oracle → couche ``raw_oracle``
- ``flow_build_processed`` — construction du modèle en étoile + scoring ML
- ``flow_publish_serving`` — publication des vues analytiques
- ``flow_oracle_only`` — **orchestration complète** : enchaîne les trois ci-dessus et trace le cycle de vie dans ``admin.pipeline_run``
- **Scripts et bibliothèques** : connecteur ``oracle_client``, chargeur ``load_raw_oracle``.
- **Outils d'automation / ordonnancement** : Prefect (planification, reprise, journalisation), déclenchables aussi depuis le **dashboard Streamlit** (« Centre de contrôle des pipelines »).

Tâches planifiées — déploiements cron opérationnels

L'automatisation ne se limite pas au déclenchement manuel : **deux déploiements Prefect planifiés** sont publiés sur le serveur d'orchestration (`scripts/serve\_scheduled\_pipeline.py`) et exécutés automatiquement selon un **calendrier cron**, sans intervention humaine.

Déploiement	Flow	CRON	Fréquence	Rôle
`pipeline-oracle-quotidien`	`flow_oracle_only`	`0 2 * * *`	Chaque nuit à 02h00	Collecte complète : extraction Oracle → processed (+ scoring ML) → serving
`serving-refresh-horaire`	`flow_publish_serving`	`0 * * * *`	Toutes les heures	Rafraîchissement des vues analytiques (léger, sans VPN)

Les deux déploiements sont à l'état **Ready**, taggés (`collecte/oracle/production`, `analytics/serving`), et Prefect **pré-calcule les prochaines exécutions** : le serveur planifie automatiquement les runs à venir (vérifié : 5 exécutions programmées — refresh horaire à 07h/08h/09h, collecte quotidienne les 18 et 19/07 à 02h00).

Le **choix des fréquences est motivé** : la collecte Oracle est lourde (~87 000 records, VPN requis) donc planifiée **de nuit** hors heures ouvrées ; le recalcul analytique est léger et sans dépendance réseau, donc **horaire** pour garder le dashboard à jour.

Preuve d'exécution. Voir capture de l'interface Prefect en annexe (déploiements, statut `Ready`, colonnes **Schedules** affichant « At 02:00 AM every day » et « Every hour every day »).

Garanties apportées par l'automatisation

Garantie	Mécanisme
Rejouabilité	Idempotence `DELETE WHERE run_id=:n` + `INSERT` — un run peut être relancé sans doublon
Traçabilité	Table `admin.pipeline_run` (flow, type, statut, horodatage début/fin)
Résilience réseau	Retry exponentiel (`tenacity`) sur les erreurs transitoires de connexion, keepalive Oracle (`expire_time`), `tcp_connect_timeout` — protège des coupures VPN sur les longues extractions
Reprise sur incident	Extraction Oracle par préfixe avec tolérance : un préfixe en échec n'invalide pas les autres

Critères couverts (C1.1.3). La méthode d'automatisation est présentée et argumentée : elle comporte ✓ des **workflows** (flows Prefect), ✓ des **scripts et librairies** (connecteurs, chargeurs), ✓

des **outils d'automation** (Prefect + déclenchement dashboard) et ✓ un **ordonnanceur** avec ✓ **tâches planifiées réelles** (2 déploiements cron actifs : quotidien `0 2 \* \* \*` et horaire `0 \* \* \* \*`, prochaines exécutions programmées automatiquement). Les garanties (rejouabilité, traçabilité, résilience) sont justifiées.

3. C1.2 — Stocker les données

C1.2.1 — Stratégie de stockage et modèle de données

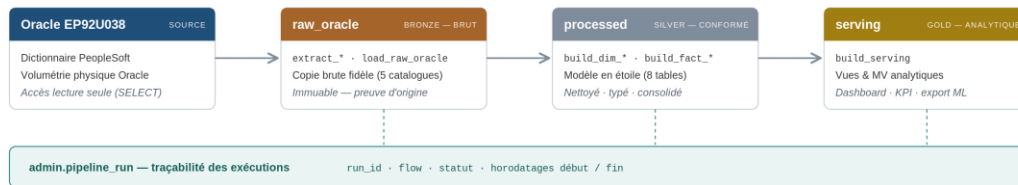
Livrables : une stratégie de stockage + un modèle de données.

Stratégie de stockage — architecture médaille

Le stockage suit le patron **médaille** (bronze/silver/gold), matérialisé par 4 schémas PostgreSQL principaux aux responsabilités séparées :

Architecture médaille — entrepôt PostgreSQL pfe_data_ia

Séparation des responsabilités par schéma - chaque couche porte le run_id



Chaque couche porte le run_id : chaque exécution est un instantané complet, rejouable et comparable (run N vs N-1).

Ce choix répond aux **usages envisagés** :

- **Disponibilité / accessibilité** : la couche `serving` (vues matérialisées) sert le dashboard et les requêtes analytiques sans recalcul.
- **Analyse** : la couche `processed` (modèle en étoile) est optimisée pour l'agrégation.
- **Stockage / auditabilité** : la couche `raw\_\*` conserve la donnée brute non transformée (rejouabilité, preuve d'origine).

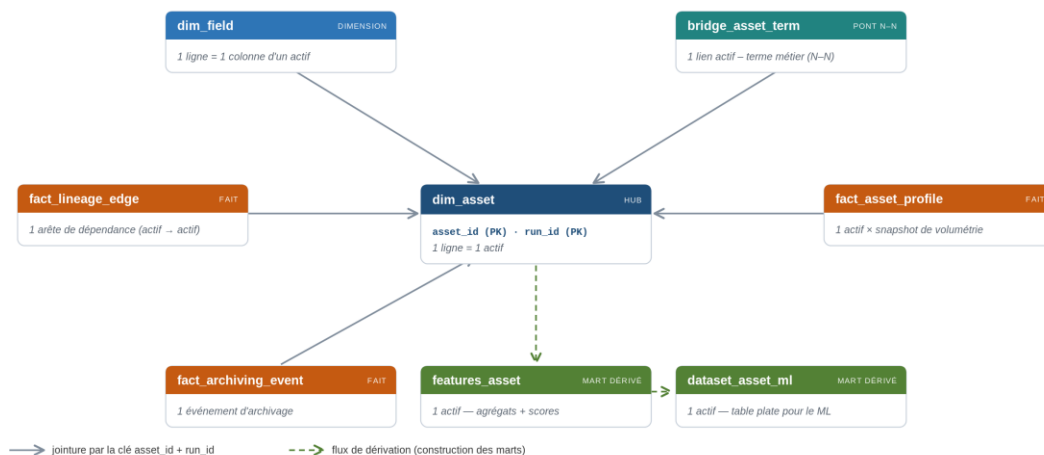
Modèle de données — schéma en étoile (couche `processed`)

La couche silver est modélisée en **schéma en étoile**, centré sur une dimension conforme unique — **l'actif de données** — adapté à l'analyse multidimensionnelle :

Modèle en étoile (constellation) — couche processed

Hub dim_asset : toutes les tables sont rattachées par la clé asset_id + run_id

■ Hub ■ Dimension ■ Pont N-N ■ Fait ■ Mart dérivé



Le run_id joue le rôle de dimension temps : chaque exécution produit un instantané complet du modèle. Détail des tables en Annexe B.3.

Toutes les branches se rattachent au hub `dim\_asset` par la clé `asset\_id` + `run\_id`. Les 8 tables :

Table	Type	Grain	Rôle
`dim_asset`	Dimension (hub)	1 actif	Entité centrale : nom technique, type, source, description
`dim_field`	Dimension	1 colonne	Champs/colonnes de chaque actif (type, nullable, longueur)
`bridge_asset_term`	Pont N-N	1 lien	Rattache un actif à des termes métier / domaine
`fact_asset_profile`	Fait	1 actif × snapshot	Volumétrie & qualité : `row_count`, `size_mb`, `column_count`, `index_count`
`fact_lineage_edge`	Fait	1 arête	Dépendances actif → actif (`lineage_type`, `lineage_level`)
`fact_archiving_event`	Fait	1 événement	Événements d'archivage (`event_type`, `rows_affected`, `volume_mb`)
`features_asset`	Mart dérivé	1 actif	Variables agrégées + scores (`archival_candidate_score`, `roi_score`)
`dataset_asset_ml`	Mart dérivé	1 actif	Table plate dénormalisée prête pour le classifieur ML

Le versionnement par ``run_id`` joue le rôle d'une dimension temps : chaque table le porte, toute requête le filtre, et chaque exécution du pipeline produit un instantané complet — d'où la **reproductibilité** et la comparaison N-1 (run 5 vs run 6).

Précision de modélisation. Au sens strict, une étoile = 1 fait + N dimensions. Ici plusieurs faits (``profile``, ``lineage``, ``archiving``) partagent la **dimension conforme** ``dim_asset`` : il s'agit donc d'une **constellation de faits** (**galaxy schema**), généralisation de l'étoile. Le détail par table et un exemple de requête figurent en **Annexe B.3**.

Critères couverts (C1.2.1). La stratégie de stockage répond aux usages exigés (✓ **disponibilité**, ✓ **analyse**, ✓ **stockage**, ✓ **accessibilité**) via l'architecture médaillon. Le **modèle de données** est décrit formellement et permet d'identifier ✓ **l'organisation des données** (schéma en étoile : dimension centrale ``dim_asset``, dimensions, faits, table de pont, clés et grain).

C1.2.2 — Construction de la base de données

Livrable : une base de données (et/ou solution Big Data).

Choix technologique

Option	Retenue ?	Justification
PostgreSQL (relationnel)	OUI — retenu	Données fortement structurées (catalogues, métriques) ; besoin de jointures, de vues matérialisées, d'intégrité ; volume (~centaines de milliers de lignes de métadonnées) parfaitement dans la cible d'un SGBDR ; open-source, gratuit, mature.
Data Warehouse cloud (BigQuery, Snowflake)	Non — écarté	Surdimensionné pour le volume ; coût et dépendance cloud non justifiés au stade prototype.
Data Lake / Big Data (Spark, HDFS)	Non — écarté	Pas de volumétrie « Big Data » ni de données non structurées massives ; complexité opérationnelle injustifiée.
Base NoSQL comme cible	Non — écarté	Inadaptée à l'entrepôt analytique (besoin de jointures et d'agrégations relationnelles sur les métadonnées).

Le choix du **SGBDR PostgreSQL** est donc justifié par la **nature relationnelle des données** et les **besoins d'analyse**, au regard des contraintes du projet.

Implémentation et paramétrage (design pattern)

La base est construite par des scripts **DDL versionnés** (``sql/ddl/``), appliqués par le flow ``flow_init_db`` :

<code>sql/ddl/001_schemas.sql</code>	→ création des schémas
<code>sql/ddl/010_admin.sql</code>	→ <code>admin.pipeline_run</code> (traçabilité)
<code>sql/ddl/020_raw_oracle.sql</code>	→ 5 tables brutes Oracle
<code>sql/ddl/040_processed.sql</code>	→ modèle en étoile (8 tables)
<code>sql/ddl/050_serving.sql</code>	→ vues et vues matérialisées analytiques

Le **design pattern médaille** est adapté au besoin : séparation stricte des responsabilités par schéma, garantissant la **disponibilité** (`serving`) et l'**intégrité** (`raw` immuable, `processed` conformé, clés et contraintes).

***Critères couverts (C1.2.2).** Le choix de la solution de stockage (✓ **système de bases de données relationnelles PostgreSQL**) est **justifié** et répond à la problématique. La base est **organisée suivant un design pattern adapté** (✓ architecture médaille + schéma en étoile), avec paramétrage et implémentation par DDL versionné garantissant disponibilité et intégrité.*

4. C1.3 — Transformer les données

C1.3.1 — Sélection des technologies et outils de traitement

Livrable : une présentation des outils et technologies de traitement sélectionnés.

Technologie	Rôle	Avantages	Inconvénients
Python 3.13	Langage de traitement	Écosystème data mature, lisibilité, large adoption	Moins rapide que Scala/C++ sur du calcul massif (non bloquant ici)
pandas	Manipulation tabulaire	API expressive, jointures/agrégations simples	Empreinte mémoire (acceptable au volume du projet)
SQLAlchemy + psycopg (v3)	Accès PostgreSQL	Requêtes paramétrées (anti-injection), pooling, portabilité	Courbe d'apprentissage
SQL (PostgreSQL)	Transformations ensemblistes (vues/MV)	Performant pour agrégation/jointure au plus près des données	Moins adapté à la logique procédurale complexe
Prefect	Orchestration ETL	Traçabilité, reprise, planification	Nécessite un serveur pour la prod
loguru / pydantic-settings / tenacity	Logs, config typée, résilience	Robustesse et configuration centralisée	—

Comparaison d'alternatives. Un traitement **Spark** a été envisagé mais écarté : le volume (métadonnées, pas données métier brutes) ne justifie pas un moteur distribué. **Scala** offrirait des performances supérieures mais au prix d'une productivité moindre et d'un écosystème ML plus

pauvre côté Python. Le couple **Python + SQL** répond **efficacement** à la problématique au regard des contraintes.

***Critères couverts (C1.3.1).** La présentation identifie ✓ les **avantages et inconvénients** des technologies (Python, SQL, pandas, SQLAlchemy, Prefect) et des **alternatives comparées** (Spark, Scala). Les technologies choisies ✓ **répondent efficacement** à la problématique énoncée.*

C1.3.2 — Transformation des données

Livrable : une présentation des données transformées, des méthodes et outils utilisés.

Les transformations conduisent de la donnée **brute** (`raw\_\*`) au modèle **conformé** (`processed`) puis aux **agrégats** (`serving`). Modules : `src/transform/`.

Transformation	Méthode	Module	Exemple
Formatage	Normalisation des noms, typage, nettoyage des espaces/valeurs nulles	`build_dim_asset`, `build_dim_field`	`TRIM`, minuscules, `NULLIF`
Consolidation	Fusion des sources logique (PeopleSoft) et physique (Oracle) en un actif unique	`build_dim_asset`	jointure record ↔ table physique
Agrégation	Calcul de métriques par actif (volume, nb lignes, taille, âge)	`build_fact_asset_profile`	`SUM`, `COUNT`, ratios
Profilage	Statistiques de qualité (ratio de nullables, présence de description)	`build_fact_asset_profile`	`avg_nullable_ratio`
Jointure	Rattachement des champs (colonnes) à leur actif parent	`build_dim_field`	jointure champ ↔ table physique
Calcul	Dérivation des variables pour le ML	`build_features_asset`, `build_dataset_asset_ml`	features texte/statistiques

Traçabilité de la transformation. Chaque table `processed` porte le `run\_id` : on peut rejouer, comparer entre runs (analyse N-1), et remonter jusqu'à la donnée brute d'origine.

***Critères couverts (C1.3.2).** La présentation identifie explicitement les transformations effectuées depuis les données sources : ✓ **formatage**, ✓ **consolidation**, ✓ **agrégation**, ✓ **profilage**, ✓ **jointure**, ✓ **calcul** — avec méthodes et outils (Python/pandas + SQL) associés.*

C1.3.3 — Processus ETL et orchestration

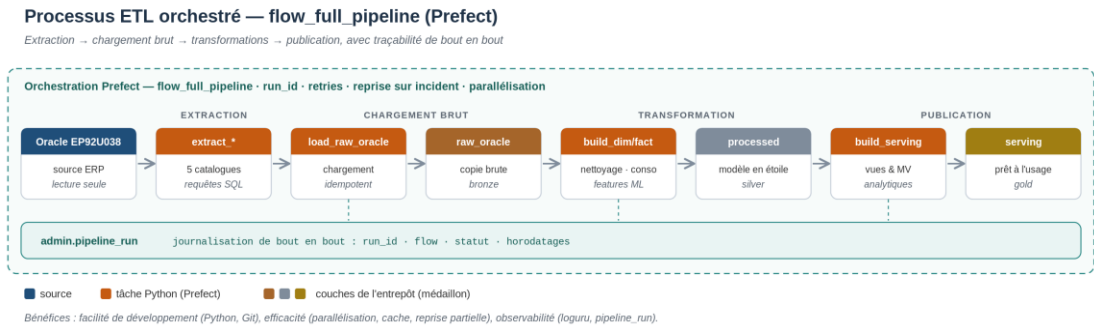
Livrable : un processus ETL + les solutions d'automatisation et d'orchestration.

Choix de la solution ETL

La solution ETL retenue est **Prefect** (orchestration Python-native), plutôt qu'un outil graphique type Talend ou SSIS.

Bénéfice attendu	Réalisation
Facilité de développement	ETL défini en Python (mêmes langage/outils que le reste du projet, versionné dans Git)
Efficacité	Parallélisation des tâches indépendantes, cache, reprise partielle
Observabilité	Journalisation structurée (`loguru`), suivi des runs en base (`admin.pipeline_run`)

Le processus ETL



Le flow ``flow_oracle_only`` **automatise et orchestre** l'enchaînement complet : création d'un ``run_id``, extraction, chargement idempotent, transformations, publication de la couche serving, avec journalisation à chaque étape dans ``admin.pipeline_run`` (statut ``success`` / ``failed``).

Critères couverts (C1.3.3). La solution ETL (**Prefect**) est **justifiée** par ses bénéfices (✓ facilité de développement, ✓ efficacité, ✓ observabilité). Le processus ETL développé ✓ **automatise et orchestre** le traitement des données (extract → transform → load, tracé par ``pipeline_run``).

5. C1.4 — Sécuriser les données

C1.4.1 — Politique de sécurité des données

Livrable : une politique de sécurité des données.

Enjeux de sécurité identifiés

- **Données personnelles (RGPD).** Les métadonnées et logs peuvent contenir des identifiants d'opérateurs, emails, adresses IP (ex. table ``PSACCESSLOG``). Enjeu : ne pas propager de PII en clair dans les exports ou le dataset de labellisation.
- **Confidentialité des accès.** Les identifiants de connexion aux bases (Oracle, PostgreSQL) sont des secrets.
- **Intégrité de la source.** Aucune écriture ne doit atteindre l'ERP de production.

Moyens mis en œuvre

Enjeu	Mesure appliquée	Emplacement
Secrets	Identifiants hors du code, dans un <code>.env</code> gitignoré ; configuration typée et validée (<code>pydantic-settings</code>) ; contrôle de non-vacuité du mot de passe Oracle	<code>`src/config/settings.py`</code> , <code>`.gitignore`</code>
Données personnelles	Masquage/pseudonymisation des PII dans les exports et le dataset de labellisation (Phase 0)	<code>`src/export/*`</code> , <code>`src/transform/build_archiving_rules.py`</code>
Chiffrement au repos	Chiffrement applicatif d'une colonne PII via <code>pgcrypto</code> (<code>pgp_sym_encrypt</code>) : la valeur est stockée en <code>BYTEA</code> illisible et n'est déchiffrable qu'avec la clé	<code>`sql/security/010_pgcrypto_demo.sql`</code> , <code>`security.pii_vault`</code>
Moindre accès à la source	Connexion Oracle en lecture seule (SELECT uniquement) ; Console SQL du dashboard bridée aux requêtes SELECT (rejet des mots-clés d'écriture)	<code>`src/connectors/oracle_client.py`</code> , <code>`app/streamlit_dashboard.py`</code>
Surveillance traçabilité /	Journalisation structurée des accès et traitements (<code>loguru</code>), historique des runs (<code>admin.pipeline_run</code>), audit trail décisionnel en base (Phase 0)	<code>`admin`</code> , <code>`serving`</code>
Sauvegarde	Couche <code>`raw_oracle`</code> immuable conservant la donnée d'origine (rejouabilité, preuve)	schéma <code>`raw_oracle`</code>
Différenciation des rôles	Comptes PostgreSQL à privilèges séparés : <code>`pfe_reader`</code> (SELECT seul) pour la consultation, <code>`pfe_writer`</code> (CRUD) réservé au pipeline ETL	<code>`sql/security/020_roles_least_privilege.sql`</code>

Chiffrement au repos des données personnelles (pgcrypto)

Le masquage protège les **exports** ; le chiffrement protège la donnée **dans la base elle-même**. L'extension `pgcrypto` chiffre symétriquement (PGP) la valeur avant stockage. Le coffre vit dans un **schéma `security` dédié** — ce n'est pas un objet analytique, il n'a donc pas sa place dans la couche `serving` (séparation des responsabilités) :

```
CREATE SCHEMA IF NOT EXISTS security;
```

```
INSERT INTO security.pii_vault (asset_id, label, email_encrypted)
VALUES (1, 'contact_responsable_finance', pgp_sym_encrypt('jean.dupont@corp.com',
:'cle'));
```

La démonstration établit **trois preuves enchaînées** :

Preuve	Requête	Résultat obtenu
Illisible au repos	<code>`SELECT email_encrypted ...`</code>	<code>`\xc30d040703024182cf37dae0345c60d24501b78d...`</code>
Déchiffrable avec la clé	<code>`pgp_sym_decrypt(email_encrypted, :'cle')`</code>	<code>`jean.dupont@corp.com`</code>
Inaccessible sans la clé	<code>`pgp_sym_decrypt(email_encrypted, 'mauvaise_cle')`</code>	<code>`ERROR: Wrong key or corrupt data`</code>

Un vol de la base (ou d'une sauvegarde) **ne suffit donc pas** à exposer la donnée personnelle : la clé est un secret distinct, détenu hors base.

Moindre privilège : comptes lecture / écriture séparés

Deux rôles PostgreSQL distincts remplacent l'usage d'un compte unique tout-puissant :

Rôle	Privilèges	Usage
<code>`pfe_reader`</code>	<code>`SELECT`</code> seul sur <code>`admin`</code> , <code>`processed`</code> , <code>`serving`</code>	Dashboard, analystes, Console SQL
<code>`pfe_writer`</code>	<code>`SELECT`</code> , <code>`INSERT`</code> , <code>`UPDATE`</code> , <code>`DELETE`</code> + séquences	Pipeline ETL uniquement

La séparation est **vérifiée par la pratique** — connecté en ``pfe_reader`` :

```
SELECT COUNT(*) FROM processed.dim_asset; → 167260
(lecture autorisée)
INSERT INTO processed.dim_asset ... → ERROR: permission denied for table dim_asset
DELETE FROM processed.dim_asset ... → ERROR: permission denied for table dim_asset
```

Le principe du **moindre privilège** est ainsi appliqué et prouvé : un compte de consultation compromis ne peut ni altérer ni détruire le patrimoine de données.

***Limites assumées.** Les mots de passe des rôles et la clé ``pgcrypto`` sont, dans cette démonstration, passés en clair aux scripts. En production, ils devraient provenir d'un **coffre à secrets** (HashiCorp Vault, Azure Key Vault) et non de la ligne de commande. De même, ``pgcrypto`` chiffre **au niveau applicatif** (colonne par colonne) : le chiffrement de l'ensemble du volume de stockage (TDE / chiffrement disque) reste une mesure d'infrastructure complémentaire.*

Conformité réglementaire (RGPD)

Le projet applique le principe de **minimisation** (on ne collecte que des métadonnées, pas le contenu métier) et de **protection par défaut** (masquage PII activé, chiffrement `pgcrypto` des valeurs personnelles au repos). Les durées de rétention légales sont modélisées par domaine (`dim\_domain\_retention`) — leur validation juridique reste une perspective assumée.

Critères couverts (C1.4.1). La politique identifie ✓ les enjeux de sécurité (PII/RGPD, secrets, intégrité) et ✓ les moyens mis en œuvre, tous opérationnels et démontrés : ✓ chiffrement des données personnelles au repos (`pgcrypto`, 3 preuves) et masquage dans les exports, ✓ surveillance (logs structurés + `admin.pipeline\_run` + audit trail), ✓ sauvegarde (couche `raw\_oracle` immuable), ✓ différenciation des rôles (comptes `pfe\_reader` / `pfe\_writer` à privilèges séparés, écriture refusée en lecture seule).

C1.4.2 — Architecture de sécurité

Livrable : un schéma d'architecture de sécurité.

Schéma d'architecture (zones, flux, protections)



Zones de sécurité et plans d'adressage

Zone	Périmètre	Adressage	Niveau de confiance
Poste analyste	Navigateur + serveur Streamlit + PostgreSQL cible	`localhost` / réseau local (`10.x`), port 8501 (UI) et 5432 (DB)	Élevé (maîtrisé)
Tunnel VPN	Canal chiffré poste ↔ réseau source	—	Transit sécurisé
Zone source	Oracle PeopleSoft EP92U038	Réseau privé `192.168.11.0/24`,	Restreint (lecture seule)

	(production)	port 1521 (Oracle)	
--	--------------	--------------------	--

Flux d'échanges de données

1. **UI ↔ serveur** : le navigateur n'échange que du texte de requête et des résultats tabulaires (Arrow) via WebSocket local — **aucun secret ni accès DB direct**.
2. **Serveur ↔ source** : extraction **lecture seule** à travers le VPN chiffré ; les identifiants restent côté serveur.
3. **Serveur ↔ cible** : écriture idempotente dans PostgreSQL par le compte `pfe\_writer` uniquement ; PII masquées dès la couche `processed` et chiffrées dans le schéma `security`. La consultation (dashboard) passe par `pfe\_reader`, qui ne peut pas écrire.

Cette architecture assure la **protection des données** : les secrets ne quittent jamais le serveur, la donnée transite chiffrée (VPN), la source est protégée en écriture, les données personnelles sont **masquées dans les exports et chiffrées en base**, et un compte de consultation compromis **ne peut ni altérer ni détruire** le patrimoine.

*Critères couverts (C1.4.2). Le schéma précise ✓ les **moyens de sécurisation** (VPN chiffré, masquage PII, **chiffrement `pgcrypto` au repos**, lecture seule, garde-fou SELECT, **moindre privilège `pfe\_reader`/`pfe\_writer`**, secrets hors dépôt), ✓ les **zones de sécurité avec plans d'adressage** (poste analyste / tunnel VPN / zone source privée), et ✓ les **flux d'échanges de données** (3 flux, avec le compte utilisé pour chacun). L'architecture ✓ **assure la protection des données en transit et au repos**.*

6. Conclusion

Ce bloc établit le socle **collecte → stockage → transformation → sécurisation** de la solution de gouvernance :

- **Collecte** fiable, exhaustive et exacte des métadonnées Oracle EP92U038, enrichie de tarifs de marché réels (API + scraping), automatisée et **planifiée** via Prefect.
- **Stockage** structuré en architecture médaillon + modèle en étoile sur PostgreSQL, choix justifié au regard du volume et des usages.
- **Transformation** traçable de la donnée brute au dataset analytique et ML, orchestrée par un ETL Python.
- **Sécurisation** de bout en bout : minimisation RGPD, masquage PII dans les exports **et chiffrement au repos** (`pgcrypto`), secrets hors dépôt, **moindre privilège** (comptes lecture/écriture séparés), accès source en lecture seule, architecture zonée avec VPN.

Ce socle alimente directement les blocs suivants : l'**analyse et la valorisation** (Bloc 2, dashboard et KPI) et la **modélisation par apprentissage automatique** (Bloc 5, classification des domaines métier).

Annexes (hors décompte de pages)

- **A.** Modules de collecte : arborescence du dépôt (src/extract/, src/connectors/, src/load/).
- **B.** Base de données : architecture médaillon, modèle en étoile détaillé, exemple de requête.
- **C.** Traçabilité des exécutions : historique des runs (admin.pipeline_run).

- **D.** Comptages de contrôle de la collecte : réconciliation source ↔ entrepôt (écart nul).
- **E.** Valorisation : captures du dashboard et export Excel.
- **F.** Sécurisation : secrets, masquage PII, lecture seule, chiffrement pgcrypto, rôles séparés.
- **G.** Requête d'extraction commentée (réconciliation logique ↔ physique).
- **H.** Collecte externe : API publique (tarifs de stockage) et web scraping (C1.1.2).
- **I.** Tâches planifiées : déploiements cron Prefect (C1.1.3).